

# Comparação de Extração de Representações Aplicado em Análise de Sentimento

1<sup>st</sup> Andre Grassi de Jesus  
Departamento de Informática (UFPR)  
Curitiba, Brasil  
agj23@inf.ufpr.br

**Resumo**—Este trabalho analisa e compara o desempenho de diferentes paradigmas de extração de representações textuais na tarefa de análise de sentimentos de avaliações cinematográficas (base IMDB). Foram avaliados os modelos Term Frequency-Inverse Document Frequency (TF-IDF), Bag-of-Words (BoW) e o Bidirectional Encoder Representations from Transformers (BERT). As características extraídas alimentaram um classificador K-Nearest Neighbors (KNN) otimizado via grid search com validação cruzada (3-fold). Os resultados indicam que o BERT atingiu a maior acurácia (80,34%), porém com um custo computacional de extração aproximadamente 300 vezes superior ao BoW, evidenciando um claro trade-off entre assertividade semântica e tempo de execução. Entre os modelos tradicionais, o BoW (75,28%) superou o TF-IDF (72,90%), demonstrando que a penalização da frequência de termos pelo IDF prejudicou a separabilidade das classes neste domínio específico.

**Index Terms**—extração de características, classificação, aprendizado de máquina

## I. INTRODUÇÃO

Um classificador é um algoritmo capaz de categorizar exemplos em classes, com base na descrição dos seus atributos [1]. Estes atributos são obtidos por meio do processo de extração de representações, que consistem em vetores numéricos [2].

Com base nisso, o objetivo do relatório é analisar e comparar representações a partir da classificação de um *dataset* de comentários sobre filmes em positivos/negativos obtidos da base IMDB. Para isso, os métodos de extração que foram utilizados são: Term Frequency-Inverse Document Frequency (TF-IDF), Bag-of-Words (BoW) e Bidirectional Encoder Representations from Transformers (BERT). As *features* extraídas por eles foram utilizadas para treinar o classificador K-Nearest Neighbors (KNN), que em seguida foi usado para classificar os comentários.

## II. REVISÃO DA LITERATURA

A extração de características pode ser feita de inúmeras formas, por exemplo pelo TF-IDF, BoW e BERT citados anteriormente.

### A. Extração de Características

1) *TF-IDF*: O método TF-IDF (Term Frequency-Inverse Document Frequency) quantifica a relevância estatística de um termo em relação a um documento pertencente a um determinado corpus, sendo formulado por:

$$\text{tf-idf}(t, d, D) = \text{tf}(t, d) \times \text{idf}(t, D) \quad (1)$$

Onde  $t$  denota o termo avaliado,  $d$  o documento específico,  $D$  o conjunto total de documentos (corpus),  $\text{tf}(t, d)$  a frequência local do termo em  $d$ , e  $\text{idf}(t, D)$  o peso da frequência inversa do documento em todo o acervo. Sob essa formulação, cada documento é projetado em um espaço vetorial, em que as dimensões correspondem aos termos do vocabulário e os valores escalares refletem seus respectivos pesos ponderados. Para mitigar a maldição da dimensionalidade e a esparsidade da matriz resultante, a cardinalidade desse espaço é delimitada previamente por um teto de atributos (*max features*), restringindo o vocabulário aos termos de maior frequência global que satisfaçam os limiares de corte documental pré-estabelecidos [3].

2) *Bag-of-Words*: O modelo Bag-of-Words é uma representação clássica, que converte documentos não estruturados em vetores numéricos de dimensão fixa  $|V|$ , correspondente à cardinalidade do vocabulário estabelecido. Dado um documento  $d$ , sua projeção vetorial  $\mathbf{x} = [x_1, x_2, \dots, x_{|V|}] \in \mathbb{R}^{|V|}$  quantifica a frequência absoluta de cada termo  $w_i \in V$  no texto, desconsiderando a ordem sintática, as relações temporais e a estrutura gramatical das sentenças [4].

3) *BERT*: Diferentemente das representações esparsas baseadas em contagem de frequências locais e globais, o modelo BERT (*Bidirectional Encoder Representations from Transformers*) fundamenta-se na arquitetura do *Encoder* do Transformer, sendo projetado para extrair representações vetoriais contínuas e sensíveis ao contexto sintático-semântico [5].

O núcleo da arquitetura baseia-se no mecanismo de autoatenção, que pondera dinamicamente a relevância relativa de cada palavra em relação a todas as demais da sentença. A bidirecionalidade estrita do modelo decorre do processamento conjunto do contexto à esquerda e à direita de cada *token* em todas as suas camadas empilhadas [5].

### B. Classificação

A classificação supervisionada tem como objetivo atribuir um rótulo categórico a uma instância não rotulada a partir de seu vetor de características. Neste trabalho, o algoritmo selecionado foi KNN. Dado um novo vetor, o modelo identifica as  $k$  amostras mais próximas no espaço vetorial do conjunto de treinamento, utilizando funções de proximidade, como a distância euclidiana ou dissimilaridade por cosseno, e designa à nova instância a classe majoritária desta vizinhança.

Além disso, como o cálculo de vizinhança é estritamente geométrico, o escalonamento e a normalização prévia dos vetores constituem uma etapa indispensável. Esse procedimento equaliza as variáveis, garantindo que características com maiores magnitudes numéricas originais não dominem desproporcionalmente o cálculo das distâncias [2].

### III. MATERIAIS E MÉTODOS

O pipeline experimental foi desenvolvido em Python 3. As representações esparsas (BoW e TF-IDF) foram implementadas utilizando os módulos `CountVectorizer` e `TfidfVectorizer` do `scikit-learn` (v1.9.0) [6]. A extração contextual densa (BERT) utilizou o modelo pré-treinado `bert-base-uncased` [5] via `AutoTokenizer` e `AutoModel` da biblioteca `transformers` (v5.16.1), com suporte de tensores provido por `torch` (v2.5.1+cu121) e `accelerate` (v1.14.0). O tratamento de dados e suporte textual apoiaram-se em `pandas` (v3.0.5), `numpy` (v2.5.2) e `nltk` (v3.10.3).

Para obter os dados e maximizar o desempenho preditivo dos modelos, o projeto implementou uma busca exaustiva de hiperparâmetros por meio do `grid search` do `scikit-learn`. O processo constrói um pipeline completo, composto pelas etapas sequenciais de: extração de características, escalonamento/normalização e então classificação (KNN).

Os hiperparâmetros manipulados para o BoW e TF-IDF foram:

- Máximo de Características: 250, 500, 1000;
- Intervalo de n-grama: (1,1), (1,2), (1,3);
- Frequência mínima: 1, 2, 3.
- Frequência máxima: 0.8, 0.9, 1.0.

Para a representação contextual (BERT), fixou-se os pesos da rede `bert-base-uncased` e variou-se o hiperparâmetro de comprimento máximo da sequência ( $L_{\max} \in \{32, 128, 512\}$ ), em que 512 *tokens* representa o limite estrutural de entrada da arquitetura.

Para a etapa de classificação, os escalonamentos utilizados foram: Maximum Absolute Value Scaling (`MaxAbsScaler`) e `Normalizer`. O espaço de busca para a otimização do classificador KNN compreendeu a variação do número de vizinhos ( $k \in \{3, 7, 11\}$ ), funções de proximidade (distância euclidiana e dissimilaridade por cosseno) e de atribuição de pesos aos vizinhos (uniform e distance).

Os dados consistem de dois arquivos textos, um para treino e outro para teste, formatados como csv, com as colunas *num* (índice), *type* (teste ou treino), *review* (opinião sobre o filme), *label* (negativo ou positivo), *file* (arquivo com o texto, não utilizado no nosso projeto). Sendo assim, são 25 mil exemplos para treino e outros 25 mil para teste.

Para o treinamento e validação, o *grid search* realiza K-Fold Cross-Validation, sendo  $k = 3$ . Dessa maneira, para cada combinação de hiperparâmetros, o algoritmo divide a base de treino em 3 blocos (*folds*) e realiza 3 rodadas: para cada rodada ele treina o KNN juntando 2 blocos e tenta prever o outro bloco. No fim das 3 rodadas calcula a acurácia média.

Em seguida, foram selecionados as melhores combinações de hiperparâmetros para cada uma das técnicas, foi realizado o treino com 100% dos dados de treino e então realizou-se a predição com os exemplos de teste, que estavam isolados até o momento.

Para a etapa de validação, foram analisadas a acurácia do classificador para cada *fold* (1, 2 e 3), bem como a acurácia média. Já no teste final, analisou-se: tempo de extração de características, tempo de predição, acurácia, precisão, recall, f1-score e matriz de confusão.

### IV. RESULTADOS

A etapa de validação cruzada determinou as configurações ótimas de hiperparâmetros para cada metodologia avaliada:

- **BERT (`bert-base-uncased`):** Comprimento máximo da sequência ( $L_{\max}$ ) de 512 tokens, pré-processamento com `MaxAbsScaler` e classificador KNN parametrizado com  $k = 11$ , métrica de similaridade de cosseno e ponderação baseada no inverso da distância.
- **Bag-of-Words (BoW):** Vocabulário limitado a 1000 atributos com n-gramas na faixa de (1,2), filtros de frequência documental  $\min\_df = 3$  e  $\max\_df = 0,8$ , associado ao `MaxAbsScaler` e KNN ( $k = 11$ , distância de cosseno e pesos por distância).
- **TF-IDF:** Espaço dimensional contido em 500 unigramas (1,1), intervalos de ocorrência  $\min\_df = 1$  e  $\max\_df = 0,9$ , normalizado via `MaxAbsScaler` e submetido ao KNN ( $k = 11$ , cosseno e ponderação por distância).

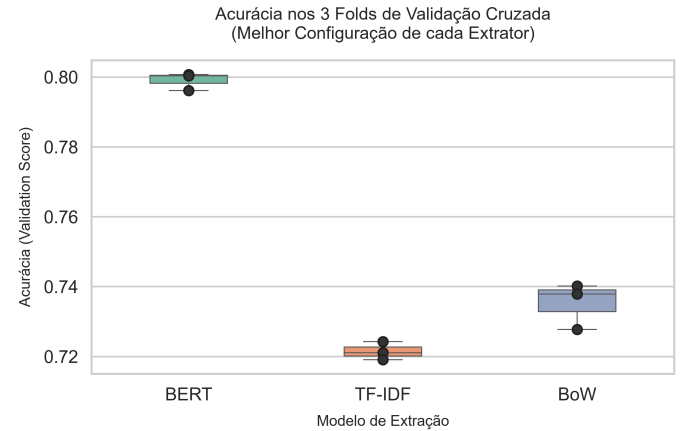


Figura 1. Boxplot dos folds na validação

Tendo isso em vista, a acurácia média na validação foi de 0.721 para o TF-IDF, 0.735 para o BoW, e 0.799 para o BERT. Na figura 1, está apresentado o box plot da combinação campeã de cada extrator e o desempenho nos 3 folds.

Em seguida, os melhores modelos foram treinados com toda a base de treino e realizaram predição com a base de teste. Os resultados podem ser observados na Tabela I.

### V. DISCUSSÃO

Os resultados evidenciam que o modelo Bag-of-Words, apesar de sua formulação mais simples, obteve acurácia superior

Tabela I  
COMPARAÇÃO DE MÉTRICAS DE DESEMPENHO

| Extrator | Tempo Ext. (s) | Tempo Pred. (s) | Acurácia | Precisão | Recall | F1-Score | Matriz de Confusão            |
|----------|----------------|-----------------|----------|----------|--------|----------|-------------------------------|
| BoW      | 20.44          | 16.14           | 0.7528   | 0.7529   | 0.7528 | 0.7527   | [[9271, 3229], [2952, 9547]]  |
| TF-IDF   | 7.64           | 15.89           | 0.7290   | 0.7292   | 0.7290 | 0.7289   | [[9298, 3202], [3573, 8926]]  |
| BERT     | 6086.72        | 31.86           | 0.8034   | 0.8074   | 0.8034 | 0.8027   | [[10755, 1745], [3169, 9330]] |

ao TF-IDF. O maior tempo de extração exigido pelo BoW justifica-se pela maior complexidade dimensional gerada pela inclusão de bigramas. A análise das matrizes de confusão revela que o TF-IDF incorreu em um aumento substancial na taxa de falsos negativos (de 2952 no BoW para 3573). Esse fenômeno indica que a atenuação de pesos imposta pelo fator IDF penalizou vocábulos de alta frequência que eram, no contexto desta base de dados, componentes discriminativos fundamentais para a separabilidade das classes.

Como esperado, a representação densa do BERT estabeleceu o melhor desempenho preditivo do experimento. Cumpre ressaltar que este modelo incorpora pesos pré-treinados que lhe conferem um vasto conhecimento linguístico a priori, justificando sua superioridade frente aos outros dois modelos. Contudo, o custo computacional de sua extração de características apresentou-se como um gargalo severo, sendo aproximadamente 300 vezes mais lento que a configuração ótima do BoW, o que compromete a viabilidade de aplicações que exijam re-extrações iterativas em larga escala. Adicionalmente, nota-se que o BERT adotou um comportamento preditivo conservador e assimétrico: mitigou expressivamente os falsos positivos (1745), mas manteve os falsos negativos elevados (3169), priorizando a precisão em detrimento do recall na identificação da classe de interesse.

Por fim, constata-se que o f1-score (média harmônica entre precisão e recall) acompanhou a acurácia global em todos os cenários. Essa simetria prova que o agrupamento do KNN foi geometricamente efetivo e não sofreu do viés da classe majoritária, o que inflaria a acurácia de forma artificial, mas provocaria uma degradação severa no f1-score.

## VI. CONCLUSÃO

Este trabalho avaliou o impacto de diferentes paradigmas de representação textual, partindo de representações mais simples (BoW e TF-IDF) até representações contextuais (BERT), no desempenho do classificador KNN para a tarefa de análise de sentimentos em avaliações de filmes. A condução de uma busca exaustiva de hiperparâmetros garantiu a otimização de cada *pipeline* antes da comparação final.

Os resultados empíricos confirmaram a superioridade semântica do BERT, que alcançou a maior acurácia do experimento (80,34%), mitigando de forma expressiva os falsos positivos. No entanto, este ganho preditivo impôs um custo computacional limitante na fase de extração, evidenciando um claro *trade-off* entre representações mais complexas e a viabilidade de tempo de execução. Entre os modelos tradicionais, o *Bag-of-Words* apresentou o melhor balanço geral: apesar de sua premissa simples, superou a configuração ótima

do TF-IDF, demonstrando que, para este domínio específico, a penalização de frequência introduzida pelo IDF descartou características relevantes para a separabilidade espacial das classes.

## REFERÊNCIAS

- [1] M. Kubat, *Introduction to Machine Learning*. Springer, 2017.
- [2] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*. John Wiley Interscience, 2001.
- [3] S. Robertson, "Understanding inverse document frequency: On theoretical arguments for idf," *Journal of Documentation*, vol. 60, no. 5, pp. 503–520, 2004.
- [4] Y. Zhang, R. Jin, and Z.-H. Zhou, "Understanding bag-of-words model: A statistical framework," *International Journal of Machine Learning and Cybernetics*, vol. 1, pp. 43–52, 12 2010.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [6] L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, R. Layton, J. VanderPlas, A. Joly, B. Holt, and G. Varoquaux, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.